

LAB – 1

8-BIT ADDITION

OBJECTIVE: To perform 8-bit Addition of two hexadecimal numbers using accumulator in 8085

PROGRAM:

Program	Explanation
MVI A, 05H	Load immediate data 05 into the accumulator
MVI B, 04H	Load immediate data 05 register B
ADD B	Add data in B to accumulator & store in the accumulator
STA C300	Store the (Result) contents of accumulator to address C300
HLT (RST 1)	End of Program

MNEMONICS:

ADDRESS	INSTRUCTIONS	OPCODE
C000	MVI A, 05	3E
C001		05
C002	MVI B, 04	06
C003		04
C004	ADD B	80
C005	STA C300	32
C006		00
C007		C3
C008	RST 1	CF

CALCULATION:

INPUT 1 = 05 = 0000 0101
INPUT 2 = 04 = 0000 0100
+ 09 = 0000 1000

INPUT 1 = 05 = 0000 0101
INPUT 2 = 02 = 0000 0010
+ 07 = 0000 0111

DISCUSSION AND CONCLUSION:

1. Instruction set used (Data transfer, Arithmetic, Logical, Branch, Program Control)
In this lab, we wrote an ALP language for 8085 to perform addition of two 8 bit numbers.
There are data transfer instructions like MVI, STA. There is an arithmetic instruction ADD.
2. Instruction set used based on size (1 byte, 2 bytes, 3 bytes) and
We have 1 byte , 2 bytes and 3 bytes instructions. One byte instructions are ADD B, HLT.
Two bytes instruction are MVI A and MVI B and one three byte instruction STA.
3. Addressing mode used (Direct, Indirect, Implicit, Immediate)
Addressing mode used are immediate (MVI), direct addressing mode(STA).
4. Hence, the program for adding two 8 – bit numbers using accumulator and register has been performed with different sets of data.

LAB – 2

8 BIT MULTIPLICATION

OBJECTIVE: To multiply two 8-bit hexadecimal numbers using 8085 microprocessor

PROGRAM:

Address/Label	Program	Explanation
	LXI H, C201	Setting pointer for data
	XRA A	Set Accumulator to '0' to account for product
	MOV B,M	Move Data 1 to Register B from Memory
	INX H	Increment the memory pointer
	MOV D,M	Move Data 2 to Register D from Memory
Repeat	ADD D	Add contents of Register D to Accumulator
	DCR B	Decrement Register B content by 1
	JNZ Repeat	If B is not Zero, got back to Repeat
	INX H	Increment the memory pointer
	MOV M,A	Store Accumulator content (Product) to memory
	INX H	Increment the memory pointer
	HLT	End of program

MNEMONICS:

ADDRESS	INSTRUCTIONS	OPCODE
C000	LXI H, C201	21
C001		01
C002		C2
C003	XRA A	AF
C004	MOV B,M	46
C005	INX H	23
C006	MOV D,M	56
C007	ADD D	82
C008	DCR B	05
C009	JNZ Repeat	C2
C00A		07
C00B		C0
C00F	INX H	23
C010	MOV M, A	77
C011	INX H	23
C012	RST 1	CF

INPUT & OUTPUT:

C201	INPUT 1	5
C202	INPUT 2	3
C203	PRODUCT	F

CALCULATION:

$$\begin{array}{rcl} & 05 & = 0000\ 0101 \\ + & 05 & = 0000\ 0101 \\ & & = 0000\ 1010 \\ + & 05 & = 0000\ 0101 \\ = & 0F & = 0000\ 1111 \\ & & = 0F\ H \end{array}$$

RESULT: The program to multiply two 8-bit hexadecimal numbers using 8085 was executed.

DISCUSSION AND CONCLUSION:

1. Instruction set used (Data transfer, Arithmetic, Logical, Branch, Program Control)
2. Instruction set used based on size (1 byte, 2 bytes, 3 bytes) and
3. Addressing mode used (Direct, Indirect, Implicit, Immediate)
4. Hence, the program for adding two 8 – bit numbers using accumulator and register has been performed with different sets of data.

Hints for Division:

Compare (register or memory) with accumulator (CMP R/M) –

If A less than (R/M), the CY flag is set and zero flag is reset.

If A equals to (R/M), the Zero flag is set and CY flag is reset.

If A greater than (R/M), the CY and Zero flag are reset.

LAB – 3

8-BIT DIVISION

OBJECTIVE: To divide two 8-bit hexadecimal numbers using 8085 microprocessor

PROGRAM:

Address/ Label	Program	Explanation
	LDA C200	Load contents of Accumulator with Divisor from memory
	MOV B,A	Move contents of Accumulator to Register B
	LDA C201	Load contents of Accumulator with Dividend from memory
	MVI C, 00	Clear register C to account for Quotient
Again	CMP B	Compare Register B with Accumulator
	JC Store	If carry==1, jump to Store
	SUB B	If carry==0, subtract Register B from Accumulator
	INR C	Increment register C by 1
	JMP Again	Jump to Again , unconditionally
Store	STA C203	Store Accumulator content (Reminder) to
		Memory

	MOV A,C	Move contents of Register C to Accumulator
	STA C202	Store Accumulator content (Quotient) to Memory
	HLT	End of Program

MNEMONICS:

ADDRESS	INSTRUCTIONS	OPCODE
C000	LDA C201	3A
		01
		C2
C003	MOV B,A	47
C004	LDA C200	3A
		00
		C2
C007	MVI C, 00	0E
		00
C009	CMP B	B8
C00A	JC Store	DA
		12
		C0

C00D	SUB B	90
C00E	INR C	0C
C00F	JMP Again	C3
		09
		C0
C012	STA C203	32
		03
		C2
C015	MOV A,C	79
C016	STA C202	32
		02
		C2
C019	RST 1	CF

INPUT & OUTPUT:

i)

C200	DIVISOR	3
C201	DIVIDEND	6
C202	REMINDER	0
C203	QUOTIENT	2

ii)

C200	DIVISOR	5
C201	DIVIDEND	9
C300	REMINDER	4
C301	QUOTIENT	1

CALCULATION:

i) Dividend A=06 ; Divisor B=03

$A - B = 06 - 03 = 03$: While $A > B$, Carry = 0, $C = C + 1$

$A - B = 03 - 03 = 00$: While $A = B$, Carry = 0, $C = C + 1$

Reminder = 0 , Carry=02, This is our Quotient.

RESULT: Thus program for dividing two hexadecimal numbers using 8085 Microprocessor has been executed

DISCUSSION AND CONCLUSION:

1. Instruction set used (Data transfer, Arithmetic, Logical, Branch, Program Control)
2. Instruction set used based on size (1 byte, 2 bytes, 3 bytes) and
3. Addressing mode used (Direct, Indirect, Implicit, Immediate)
4. Hence, the program for adding two 8 – bit numbers using accumulator and register has been performed with different sets of data.

LAB – 4

ADDITION OF 'n' NUMBERS IN ARRAY

OBJECTIVE: To add 'n' numbers in array and find the sum of those numbers using 8085 Microprocessor with memory pointer.

PROGRAM:

Address	Program	Explanation
	LXI H, C201	Setting pointer for data (No. of data in array)
	MOV B,M	Move number of data to Register B from Memory
	MVI C, 00	Clear register C to account for Carry
	XRA A	Set Accumulator to '0' to account for product
Repeat	INX H	Increment the memory pointer
	ADD M	Add Contents of Memory to Accumulator & store in Accumulator
	JNC Ahead	If carry==0 , go Ahead
	INR C	Increment register C by 1, If carry ==1
Ahead	DCR B	Decrement Register B content by 1
	JNZ Repeat	If B is not Zero, got back to Repeat
	STA C300	Store Accumulator content (Sum) to Memory
	MOV A,C	Move contents of Register C to Accumulator
	STA C301	Store Accumulator content (Carry) to Memory
	HLT (RST 1)	End of Program

MNEMONICS:

ADDRESS	INSTRUCTIONS	OPCODE
C000	LXI H, C201	21
		01
		C2
C003	MOV B,M	46
C004	MVI C, 00	0E
		00
C006	XRA A	AF
C007	INX H	23
C008	ADD M	86
C009	JNC Ahead	D2
		0D
		C0
C00C	INR C	0C
C00D	DCR B	05
C00E	JNZ Repeat	C2
		07
		C0
C011	STA C300	32
		00
		C3
C014	MOV A,C	79

C015	STA C301	32
		01
		C3
C018	RST 1	CF

INPUT & OUTPUT:

C200	No. of Elements	6
C201	DATA 1	2
C202	DATA 2	4
C203	DATA 3	6
C204	DATA 4	8
C205	DATA 5	0A
C206	DATA 6	0C
C300	TOTAL	2A
C301	CARRY	0

CALCULATION:

DATA 1 : 02 - 0000 0010

DATA 2 : 04 - 0000 0100

 + 06 - 0000 0110

DATA 3 : 06 - 0000 0110

	+	0C	- 0000 1100
DATA 4 :		08	- 0000 1000
	+	14	- 0001 0100
DATA 5 :		0A	- 0000 1010
	+	1E	- 0001 1110
DATA 6 :		0C	- 0000 1100
TOTAL :		2A	- 0010 1010
CARRY :		00	

RESULT: Program to add 'n' numbers in array and find the sum of those numbers using 8085 Microprocessor with memory pointer was executed.

DISCUSSION AND CONCLUSION:

1. Instruction set used (Data transfer, Arithmetic, Logical, Branch, Program Control)
2. Instruction set used based on size (1 byte, 2 bytes, 3 bytes) and
3. Addressing mode used (Direct, Indirect, Implicit, Immediate)
4. Hence, the program for adding two 8 – bit numbers using accumulator and register has been performed with different sets of data.

LAB – 5

LARGEST IN ARRAY

AIM: To find the Largest element in an array of size 'n' using 8085 Microprocessor.

PROGRAM:

Address	Program	Explanation
	LXI H, C200	Setting pointer for data
	MOV B,M	Move number of data to Register B from Memory
	INX H	Increment the memory pointer
	MOV A,M	Move contents of memory to Accumulator
	DCR B	Decrement Register B content by 1
Loop	INX H	Increment the memory pointer
	CMP M	Compare contents of memory with Accumulator
	JNC Ahead	If carry==0, go Ahead i.e., number is smaller
	MOV A,M	Move contents of memory to Accumulator
Ahead	DCR B	Decrement Register B content by 1
	JNZ Loop	
	STA C300	If Register B is not equal to '0', go to Loop
	HLT (RST 1)	Store Accumulator content (largest number) to Memory

MNEMONICS:

ADDRESS	INSTRUCTIONS	OPCODE
C000	LXI H, C200	21
		00
		C2
C003	MOV B,M	46
C004	INX H	23
C005	MOV A,M	7E
C006	DCR B	5
C007	INX H	23
C008	CMP M	BE
C009	JNC Ahead	D2
		0D
		C0
C00C	MOV A,M	7E
C00D	DCR B	5
C00E	JNZ Loop	C2
C011	STA C300	32

		00
		C3
C014	RST 1	76

INPUT & OUTPUT:

C200	No. of Elements	6
C201	DATA 1	04
C202	DATA 2	03
C203	DATA 3	01
C204	DATA 4	22
C205	DATA 5	05
C206	DATA 6	0C
C300	TOTAL	01

CALCULATION:

No. of Data =B =6

A=04 ; B=5

M=03 , Compare M with A: Carry=1 B=4

M=01, Compare M with A: Carry=1 ; B=3

M=22, Compare M with A: Carry=0,

A=22 ;B=2

M=05, Compare M with A: Carry=1, B=1

M=0C, Compare M with A: Carry=1, B=0

Hence, A=22 Largest number

DISCUSSION AND CONCLUSION:

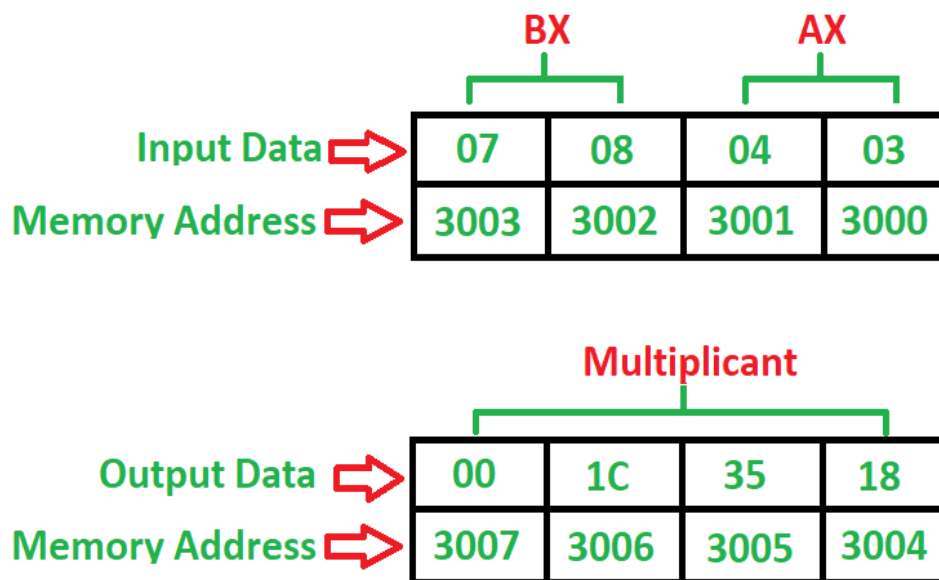
1. Instruction set used (Data transfer, Arithmetic, Logical, Branch, Program Control)
2. Instruction set used based on size (1 byte, 2 bytes, 3 bytes) and
3. Addressing mode used (Direct, Indirect, Implicit, Immediate)
4. Hence, the program for adding two 8 – bit numbers using accumulator and register has been performed with different sets of data.

LAB – 6 (8086)

MULTIPLICATION OF TWO 16-BIT NUMBERS

OBJECTIVE: To multiply 2 numbers (16-bit data) where starting address is 2000 and the numbers are at 3000 and 3002 memory address and store result into 3004 and 3006 memory address using 8086 Microprocessor.

Example –



Algorithm –

1. First load the data into AX(accumulator) from memory 3000
2. Load the data into BX register from memory 3002
3. Multiply BX with Accumulator AX
4. Move data from AX(accumulator) to memory
5. Move data from DX to AX
6. Move data from AX(accumulator) to memory
7. Stop

Program –

Memory	Mnemonics	Operands	Comment
2000	MOV	AX, [3000]	[AX] <- [3000]
2004	MOV	BX, [3002]	[BX] <- [3002]
2008	MUL	BX	[AX] <- [AX] * [BX]
200A	MOV	[3004], AX	[3004] <- AX
200E	MOV	AX, DX	[AX] <- [DX]
2010	MOV	[3006], AX	[3006] <- AX
2014	HLT		Stop

Explanation –

1. **MOV** is used to load and store data.
2. **MUL** is used to multiply two 16-bit numbers.
3. **HLT** is used to stop the program.
4. **AX** is an accumulator which is used to store the result.
5. **BX, DX** are general purpose registers where BX is used for multiplication and DX is used for result.

INPUT & OUTPUT:

Input 1 = dW = 16 bit number = 0403

Input 2 = dW = 16 bit number = 0708

Result = 001C 3518

3000	Input 1	03
3001		04
3002	Input 2	08
3003		07
3004	Result was in AX	18
3005		35
3006	Result was in DX	1C
3007		00

LAB – 7 (8086)

DISPLAY A STRING '**Electrical and Electronics Engineering**' using 8086.

Title Display '**Electrical and Electronics Engineering**'

Dosseg

.model small

.stack 100h

.data

String1 db 'Electrical and Electronics Engineering', '\$'

.code

Main proc

MOV AX, @data

MOV DS, AX

MOV AH, 09H

MOV DX, offset String1

INT 21H

MOV AH, 4CH

INT 21H

Main endp

End Main

DISCUSSION AND CONCLUSION:

A string is declared in the .DATA section. The name of the string is set to

aString and the type of the data is byte (DB=data byte). Although it is not only a single byte, assemblers are only concerned with the data type of the first item, which is then assumed for all other declarations before a new label. At the end of the string is a 13 and 10 - these are the carriage return and linefeed characters that are used to go to the next line. The last '\$' is needed by the output function to signal the end of the string.

Before the program can use the data in the data segment, the DS register must first be set up appropriately. Unlike the CS register which is always set when the program starts, the DS register must be explicitly set to point to the DATA segment. The first "mov" command gets the SEGment of the DATA segment and stores it in the AX register. The second "mov" command sets the DS register value from the AX register. The reason why two commands are necessary is because the 8086 CPU does not have a command to directly set a value into a segment register.

The value of 9 is inserted into the AH register to select sub-function 9 of the interrupt 21h DOS interrupts. This interrupt requires that the DS:DX segment:offset pair point to the string to be output. In this case, DS already points to the segment containing the string. So we just set the DX register to the OFFSET of the string.

Interrupt 21h is called to output the string and the program terminates like before.